Fragile Authentication of PDF Signatures via Bit-Exact Quantization Index Modulation on Vector Geometry

Manuscript for IEEE Transactions on Information Forensics and Security — content draft (IEEE two-column template applied at submission)

Yaroslav V. Baienko

Index Terms — fragile watermarking; quantization index modulation; vector graphics; PDF signatures; document authentication

Abstract

A PDF signature verifies the bytes it covers, not the page a human sees. The visible layer — seals and stamps — is not cryptographically bound to the signed content, so a valid signature can accompany altered or transplanted visual material. We study a fragile-authentication watermarking approach that closes this gap at the rendering level: the digest of the signed document is embedded, bit-exactly, into the geometry of the visible seal, so the mark a person sees is also the mark that is verified. The carrier is a planar incidence configuration of nine triangles (27 degrees of freedom); the payload uses keyed dithered quantization index modulation over an integer coordinate lattice with an IEEE-754 rounding argument, and a disjoint degree of freedom carries a per-recipient identifier. No external QR/barcode is added. We evaluate 180 PDFs from 60 documents (LibreOffice, Typst and Chromium), in native, flattened and print-scan-simulated variants, plus ten project and five independent public documents. On 1,800 stamped PDFs the 266-bit payload is recovered exactly, survives 8-decimal re-serialisation, fails closed at 7 decimals, and detects every edited payload coordinate; flattening destroys the vector carrier and fails closed. Same-carrier baselines expose a capacity–distortion trade-off. Two implementation facts: the PDF writer must preserve at least nine decimal places, and the verifier must parse coordinates as text — 32-bit-float drawing APIs destroy the 1.8×10^{-8} pt lattice. A print-scan check fails closed in all five cases; digital controls verify exactly. It complements, not replaces, PAdES/eIDAS signatures.

1. Introduction

A signed PDF can be **valid** and still mislead. The signature covers a byte range; the visual seal is an image or a vector drawing that the signature does not necessarily cover, and incremental updates or flattened copies live outside the signed bytes. The literature on PDF signature attacks documents hidden content ([1]) and signature spoofing ([2]), and document workflows add a third problem: a data room can hand every recipient a visually identical copy, so a leak cannot be attributed.

This paper asks a watermarking question rather than a cryptographic one: can a fragile mark be carried by the visible layer itself, so that (i) the mark is exactly recoverable from the rendered page, (ii) any modification of the mark is detected, and (iii) the mark identifies the recipient copy? The answer explored here is a bit-exact quantization-index-modulation payload embedded in the coordinates of a vector seal. The word **fragile** is deliberate: the goal is not robustness to editing but detection of it.

The contributions of this draft are: (1) a bit-exact fragile payload in vector geometry with an exact rounding bound; (2) a carrier that uses the seal itself, so removing or transplanting the seal breaks verification; (3) a disjoint recipient channel; and (4) an evaluation protocol with same-carrier baselines, three document generators, flattened and print-scan variants, and a carrier ablation. Section 2 positions the work against the watermarking literature, Section 3 states the threat model, Section 4 describes the carrier and the bit-exact embedding, Section 5 reports the PDF-level evaluation, Section 6 gives the attack matrix and the limits of the security argument, and Section 7 discusses threats to validity and reproducibility.

Prior versions. Earlier editions of this research line were posted as preprints ([3]). This manuscript is a substantially re-framed and expanded version: it narrows the claim to fragile authentication and adds the PDF-level evaluation, same-carrier baselines, writer precision and physical print-scan; it is not a duplicate of those editions.

2. Related Work

Family	Purpose	Carrier	What it optimises
Spread spectrum ([4])	robustness	pixels / coefficients	survival under attacks
QIM and dither modulation ([5]; [6]; [7])	rate–distortion	scalar samples	capacity at a distortion budget
RDH ([8]; [9]; [10]; [11]; [12])	reversibility	pixels / vertices	exact cover recovery
Vector-graphics watermarking ([13]; [14]; [15]; [16])	robustness	2D geometry	survival under rescaling
PDF watermarking ([17])	document marking	text/objects	blind detection
This work	fragility	visible seal geometry	bit-exact digest recovery

Positioning: the paper borrows the QIM lattice but targets fragility, and uses the visible seal rather than an existing drawing as the carrier.

2.1. Fragile versus robust watermarking

Robust watermarking aims to survive processing; fragile watermarking aims to detect it. Our work belongs to the second class and is therefore not comparable to robust schemes on BER under compression — the meaningful axes are exact recovery, distortion and detection. The QIM family ([5]; [6]; [7]) supplies the lattice; spread-spectrum schemes ([4]) and early data-hiding constructions ([18]) supply the historical contrast. Watermarking security analysis ([19]) and QIM steganalysis ([20]) frame what a keyed dither can and cannot promise.

The QIM literature is mature on rate–distortion: dither modulation, distortion-compensated QIM and spread-transform variants trade embedding rate against distortion under an additive-noise model ([5]; [6]). Our setting differs in two ways that matter for the design. First, the channel is **fragile**: the verifier must reject any change, so there is no noise budget to optimise against and the relevant question is exactness, not robustness. Second, the host is a vector drawing whose coordinates are serialised as decimal text, so the lattice step must survive decimal formatting and parsing, not an analog channel. The steganographic-security side of QIM ([19]; [20]) is also relevant but not claimed: the keyed dither prevents an unkeyed reader from aligning the payload, and a formal security claim is left out of scope.

2.2. Reversible data hiding

Difference expansion ([8]), histogram shifting ([9]) and prediction-error expansion ([10]) define the RDH family. On a 27-slot carrier these schemes are capacity-limited, which we report rather than hide; 2D vector-map RDH ([11], [12]) operates on much larger vertex sets, where capacity is not the binding constraint.

2.3. Vector graphics and PDF watermarking

Fourier-descriptor watermarking of 2D vector graphics ([13]), blind vector-graphics schemes ([14], [15]), polar-coordinate vector-data watermarking ([16]) and mesh-spectral maps ([21]) are the closest carrier-level relatives. PDF watermarking itself has been studied in [17], and print-scan resilience in [22]. Our carrier is unusual in that the mark is a **visible seal**, not a hidden embedding in an existing drawing; this is what makes the transplant attack meaningful.

2.4. PDF signature security and standards

Shadow attacks ([1]), signature spoofing ([2]), certification attacks ([23]) and practical decryption exfiltration ([24]) establish the threat model. Standards ([25], [26], [27]) define what a compliant signature does and does not cover. Our method does not change the signature; it makes the visible layer that the signature leaves uncovered tamper-evident.

2.5. Commitments, fingerprinting, terminology

Vector commitments ([28], [29]) and traitor tracing ([30], [31]) are related but not claimed here: the construction is a watermark, not a commitment scheme, and the recipient channel is attribution, not collusion-resistant tracing. The paper deliberately avoids “vector commitment” and “traitor tracing” as descriptions of its own mechanism.

2.6. Geometric background and visible seals

The nine-triangle incidence configuration has been studied as a mathematical object ([32]; [33]), and projective incidence algebra is standard ([34]); the paper uses the configuration only as a stabilised carrier and does not claim a new geometric theorem. Floating-point exactness follows the standard treatment ([35]). A related industrial idea is the visible digital seal: a QR or bar-code symbol printed next to a document and bound to its content. Such a seal

is an **external** symbol that can be detached or replaced; the present work removes the symbol and lets the drawing itself carry the payload, which is what makes the transplant attack meaningful.

2.7. What is new, and what is not

The paper does not claim a new modulation family: QIM, dither modulation and the capacity–distortion view are classical ([5]; [6]; [7]), and RDH ([8]; [9]; [10]) and vector-graphics watermarking ([13]; [14]; [15]) are established. The claim is narrower: a **visible seal** can carry a 256-bit digest bit-exactly, which turns a fragile watermark into a signature-to-rendering binding; the same carrier provides a geometric invariant and a disjoint recipient channel; and the implementation precondition (nine-decimal writers, text-level parsing) is made explicit. Equally, the paper does not claim cryptographic security from a watermarking argument, does not use "vector commitment" or "traitor tracing" as descriptions of its mechanism, and does not claim robustness: robustness is the opposite of the intended behaviour.

3. System Model and Notation

We consider a document d with a canonical byte representation and a visible seal S drawn as a planar collection of triangles. The issuer computes a 256-bit digest $h = H(d)$ and a 10-bit recipient word r, and embeds (h, r) into the seal geometry with a secret key k. A verifier receives a PDF and a claimed digest h' (from the same document) and returns a state in $\{\mathrm{valid}, \mathrm{modified}, \mathrm{absent}\}$. The carrier is described by 27 scalar slots $\boldsymbol{s} = (s_1, ..., s_{27})$ in page coordinates; the word map $\delta : [0, 1023] \to \mathbb{R}$ and the key schedule (π_K, d_K) are defined in Section 4. Table 1 lists the notation used below.

Symbol	Meaning
d, $h = H(d)$	document and its 256-bit digest
r	10-bit recipient identifier
$\boldsymbol{s}$	27 carrier slots in page coordinates
$\delta(w)$	lattice shift for a 10-bit word, step 18 npt
π_K, d_K	keyed slot permutation and word dither
$\frac{\text{valid}}{\text{modified}}$ absent	verifier output states

Notation. The three verifier states are mutually exclusive and "absent" is never mapped to "valid".

4. Threat Model and Requirements

The adversary can (a) edit the PDF content stream, (b) re-serialise coordinates at reduced precision, (c) flatten the page to a raster image, (d) transplant a visual seal from another document, and (e) redistribute a copy. The verifier knows the canonical seal geometry and the document digest. We state four requirements:

1. **Exactness.** The 256-bit digest is recovered bit for bit; a single wrong bit is a failure.
2. **Fragility.** Any edit of a payload coordinate changes the recovered digest.
3. **Fail-closed.** When the vector layer is absent (flattening), the verifier refuses rather than accepts.
4. **Attribution.** A recipient identifier is carried independently of the digest.

Definition 1 (exactness). For every document d, `Verify(Stamp(d, k), d, k) = valid`, and the recovered 256-bit payload equals the digest of d bit for bit.

Definition 2 (fragility). If any coordinate of the carrier path is modified, `Verify` returns `modified` — either because the recovered digest differs from the document digest, or because a geometric invariant fails.

Definition 3 (fail-closed). If the carrier path is absent (e.g. the page was rasterised), `Verify` returns `absent`. It never returns `valid` on a page whose carrier cannot be parsed.

Definition 4 (attribution). With the key, `Verify` returns the 10-bit recipient identifier; with a different key, the recovered identifier is not a valid attribution (the expected behaviour is a random-looking word, not the true identifier).

The threat model is deliberately weaker than a cryptographic one: we do not claim that an adversary cannot produce a new document with a new seal, only that an edited or transplanted seal is detectable relative to a known document digest.

5. Method

5.1. Carrier and degrees of freedom

The seal is drawn as nine triangles. Each triangle contributes three degrees of freedom — base line, apex, half-width — for 27 scalar slots. Because the triangles share a construction centre, the bilateral relation $lx + rx = 2cx$ and the common centre are geometric invariants, checked independently of the payload. A generic 27-vertex polyline and a 9×3 grid are used as ablations: they carry the same 27 slots at the same distortion, but have no such invariant.

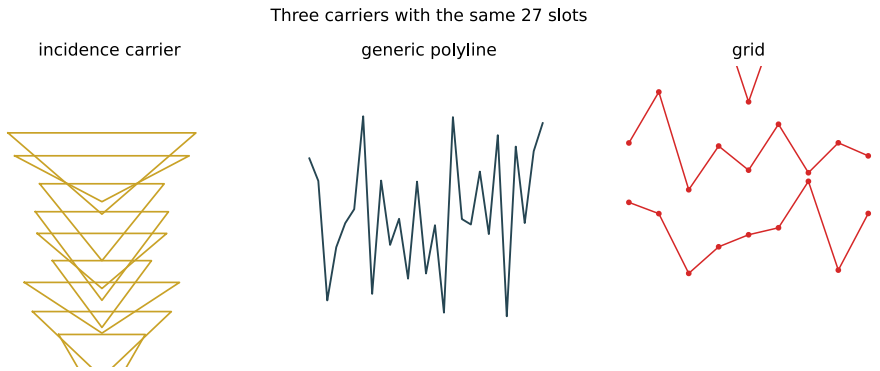


Figure 1: The three carriers used in the ablation. All three carry the same 27 slots at the same distortion; only the incidence configuration (left) has the bilateral and common-centre invariants.

5.2. Bit-exact word mapping

A 256-bit digest is split into 26 words of 10 bits; the 27th word carries a 10-bit recipient identifier. A 10-bit word w maps to the coordinate shift

$$\delta(w) = (18w - 9207) \times 10^{-9} \text{ pt},$$

i.e. a multiple of 18 npt $= 1.8 \times 10^{-8}$ pt, with $w \in [0, 1023]$. The smallest step is therefore 1.8×10^{-8} pt.

5.3. Exactness argument

At coordinates of the order of 3×10^2 pt the IEEE-754 double spacing is $2^{-52} \times 2^9 \approx 1.1 \times 10^{-13}$ pt, more than four orders of magnitude below the step, so every lattice point is exactly representable and a recovered word has no rounding ambiguity. Re-serialising a coordinate with eight decimal places rounds it to the nearest 10^{-8} pt, an error of at most 5×10^{-9} pt, below half a step (9×10^{-9} pt); seven decimal places (error up to 5×10^{-8} pt) exceeds half a step and destroys the word. This bound is the reason the evaluation attacks truncation at 8 and 7 decimals rather than at arbitrary precision.

Lemma 1 (the word map is a bijection). *For $w \in [0, 1023]$, $\delta(w) = (18w - 9207) \times 10^{-9}$ takes 1,024 distinct values in steps of 18×10^{-9} pt; decoding by $w' = \left\lfloor \frac{\lfloor \delta \cdot 10^9 \rceil + 9207}{18} \right\rceil$ returns w for every value.* ***Proof sketch:*** *the map is affine and injective on the integers, and the spacing between neighbouring values is 18 integer nanometres, so the rounding in the decoder cannot cross a boundary.*

Lemma 2 (eight decimals are sufficient). *Re-serialising a coordinate to $d = 8$ decimal places produces error at most 0.5×10^{-8} pt; since half a step is 9×10^{-9} pt, the decoder still recovers the same word. At $d = 7$ the error bound is 5×10^{-8} pt, above half a step, and recovery is no longer guaranteed.*

5.4. Keyed dither and recipient channel

A key derives a permutation π_K of the 27 slots and a word-level dither d_K. The digest words are dithered and permuted before embedding, and the recipient word is placed in the remaining slot. The dither does not change the lattice spacing, so exactness is preserved; it changes which physical slot carries which word, so an unkeyed reader cannot align the payload with the geometry. The recipient channel is written in the same way: a 10-bit identifier is mapped to a shift and placed in the 27th slot.

5.5. Verification algorithm

Input: PDF page, canonical geometry, document digest, key.

1. Parse the content stream as text and locate the carrier path (unique stroke colour/width).

2. Convert the coordinate strings to double precision; reconstruct the 27 slots from the path.
3. Undo the permutation and dither with the key; recover the 27 words.
4. Recompute the digest of the signed document and compare it with the recovered 256-bit digest.
5. Check the geometric invariants (bilateral symmetry and common centre) for the incidence configuration.
6. If the carrier is absent, return **absent** (fail-closed), not **valid**.

5.6. Encoding and decoding

Encode (d, r, k):
1. $h \leftarrow H(d)$; split h into 26 ten-bit words $w_1..w_{26}$; append $w_{27} \leftarrow r$.
2. derive (π_K, d_K) from the key and a document nonce; $v_i \leftarrow \left(w_{\pi_{K(i)}} + d_{K(i)}\right) \bmod 1024$.
3. $s'_i \leftarrow s_i + \delta(v_i)$ for $i = 1..27$.
4. write the path operators with nine decimals; return the stamped PDF and h.

Decode (PDF, h, k):
1. parse the content stream as text; locate the carrier path by marker colour/width.
2. reconstruct s'_i from the path; $v_i \leftarrow \delta^{-1}(s'_i - s_i)$; undo (π_K, d_K).
3. reassemble the 27 words; compare words 1–26 with $H(d)$; return the recipient word.
4. if the path is absent: return “absent”; if comparison fails: “modified”; else “valid”.

5.7. PDF implementation precondition

The carrier must be written with at least nine decimal places. Common PDF writers emit five decimals, which destroys the 1.8×10^{-8} pt lattice; the reference implementation therefore injects path operators into the content stream directly rather than using a drawing API. The verifier must parse the numbers as text, because drawing APIs that store coordinates as 32-bit floats lose the lattice as well (a drawing API returns coordinates quantised at about 10^{-5} pt, i.e. three orders of magnitude coarser than the step).

5.8. Implementation details

The reference implementation is Python with PyMuPDF for page handling; the carrier is injected into the page content stream as path operators formatted with nine decimals, not drawn through the shape API. The carrier path is identified by a unique stroke colour and width, which keeps the verifier independent of the page content. The 256-bit digest is the SHA-256 of the canonical document bytes; the key schedule is derived from a secret and a document nonce with SHA-256, producing the 27-slot permutation and the word-level dither. Verification returns one of three states — **valid**, **modified**, **absent** — and never maps **absent** to **valid**. Multi-page documents are stamped page by page if a per-page seal is desired; the digest binds the canonical document bytes, so a seal on any page verifies the whole document. Recipient identifiers are assigned as 10-bit words, which bounds a deployment to 1,024 tracked copies per key epoch; larger deployments rotate the key or extend the identifier space, which changes the payload budget and is out of scope here.

5.9. Complexity and deployment notes

Stamping is linear in the number of pages and constant in payload length: the carrier is a fixed 27-slot path, so the cost is one content-stream edit per page. On the development machine stamping costs about 1.05 ms and verification about 0.33 ms per page, both dominated by PDF parsing and stream rewriting rather than by the embedding arithmetic. A deployment assigns 10-bit recipient identifiers, which bounds one key epoch to 1,024 tracked copies; larger deployments rotate the key or extend the identifier space, which changes the payload budget. The verifier needs the canonical seal geometry and the key; it does not need the original PDF, only the document digest, so a recipient can verify a copy without storing the issuer’s document. Key management, revocation and the legal status of the generated evidence are deployment questions outside the watermarking scope of this paper.

6. Evaluation

6.1. Corpus

Sixty synthetic documents are generated by three independent pipelines: LibreOffice (DOCX to PDF), Typst, and Chromium print-to-PDF. Each document exists in three variants: **native**, **flattened** (300 dpi raster re-wrapped as an image-only PDF) and **print-scan simulated** (200 dpi, Gaussian blur, sensor noise, JPEG q=70). The corpus contains 180 PDFs (51.1 MB) with a SHA-256 manifest.

Generator	Documents	Variants	Pages (native)	Size
LibreOffice DOCX to PDF	20	native / flattened / print-scan	2	18.4 MB
Typst	20	native / flattened / print-scan	2	20.3 MB
Chromium print-to-PDF	20	native / flattened / print-scan	1	12.5 MB
Total	60	180 PDFs	—	51.1 MB

Corpus composition. The carrier is a visible seal of about 126×185 *pt (23,227 pt²), i.e. 266 payload bits correspond to* 0.0115 *bit/pt² of visible seal area.*

6.2. Protocol

For each document and seed, the payload is H(doc_sha ‖ seed); the 266-bit payload is embedded with keyed QIM, and the stamped PDF is attacked in the content stream (8/7-decimal truncation, payload-coordinate edit, centre-line edit) or flattened. The carrier is extracted by parsing the stream. The experiment runs 60 documents × 30 seeds (1,800 samples), plus 360 samples on flattened/print-scan inputs, plus an ablation over three carriers and keyed/unkeyed embedding.

6.3. Metrics and attack parameters

Geometric RMSE is the root-mean-square coordinate change over the 27 slots; **PSNR_geom** is $20 \log_{10}(300 \ / \ \mathrm{RMSE})$ with the 300 pt geometry normalisation. **Exact** is one only if all 27 recovered words equal the embedded words; otherwise the sample contributes a bit-error rate over 27 words. **Detection** is one if the recovered words differ from the expected words (payload edit) or if a geometric invariant fails (centre-line edit). Truncation rewrites every coordinate number in the carrier path to 8 or 7 decimal places. The payload-coordinate edit changes one coordinate by 10^{-4} pt; the centre-line edit changes a non-payload coordinate by 0.5 pt. Flattening replaces the page by a 300 dpi raster image. All attacks are applied to the stamped copy; the flattened/print-scan corpus variants are inputs to the protocol, not attacks on a stamped document.

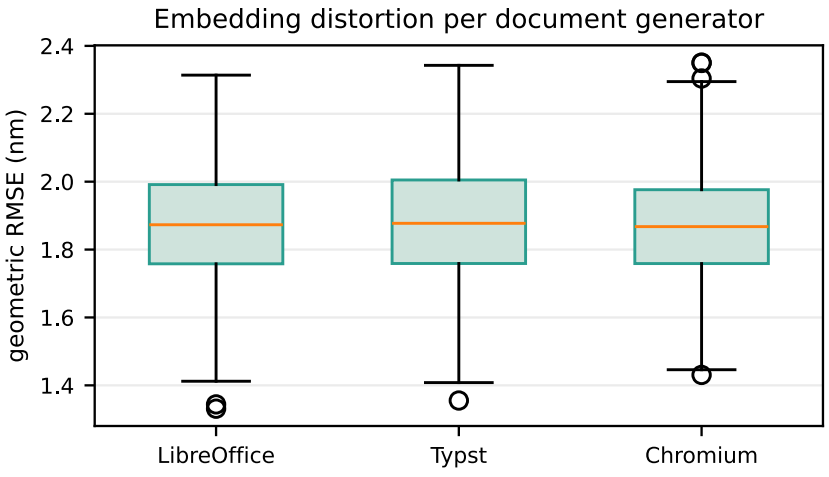


Figure 2: Embedding distortion per document generator (geometric RMSE in nanometres; medians 1.87–1.88 nm).

6.4. Precision precondition in practice

The exactness argument predicts a cliff between eight and seven decimal places. The measurement matches the prediction exactly: with the injected carrier kept at nine decimal places, exact recovery is 100% at 9 and 8 decimals and drops to 0% at 7 and 6 decimals (Figure 3). This is a property of the **pipeline after sealing**, not of the document

generator: the corpus generators themselves emit between three and six decimal places (LibreOffice about 3.2, Chromium about 5.3, Typst about 5.8 on average), and the carrier is injected after generation. A production document management system that re-writes coordinates with fewer than eight decimals therefore destroys the mark; the verifier reports **absent** rather than **valid** in that case (fail-closed).

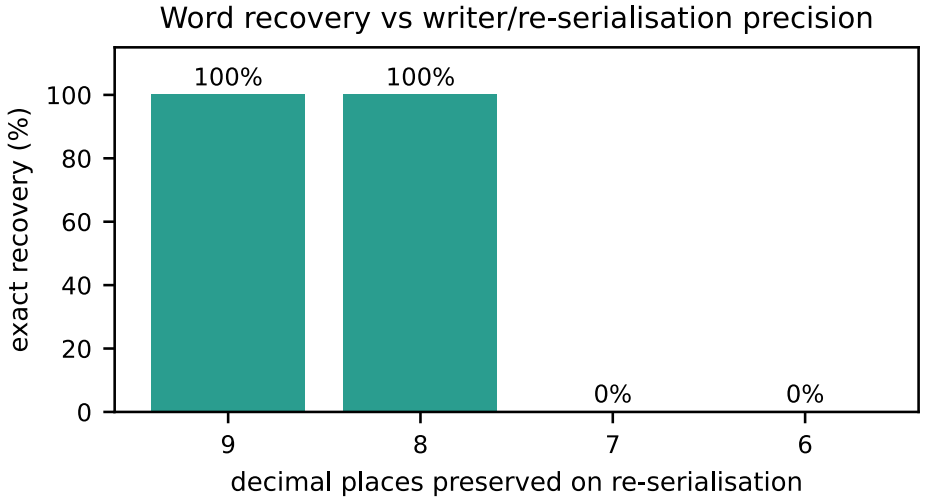


Figure 3: Word recovery versus the number of decimal places preserved on re-serialisation. The transition between eight and seven decimals is the predicted half-step boundary.

Generator	Average decimal places in native PDF	Carrier exact after injection
LibreOffice DOCX to PDF	3.16	100%
Chromium print-to-PDF	5.27	100%
Typst	5.76	100%

Writer precision of the corpus generators. The carrier is injected after generation at nine decimals; the relevant risk is a later re-writing stage.

6.5. Experimental questions

The evaluation is organised around four questions. **RQ1 (exactness):** is the 266-bit payload recovered bit-for-bit on documents from independent generators? **RQ2 (precision):** at which decimal precision does re-serialisation destroy the payload, and does the measured cliff match the half-step prediction? **RQ3 (fragility):** are coordinate edits and non-payload geometric edits detected, and does flattening fail closed? **RQ4 (carrier contribution):** with capacity and distortion held fixed, does the incidence configuration contribute anything beyond a generic polyline or a grid? Each question maps to a table below; no claim in the abstract goes beyond them.

6.6. Results

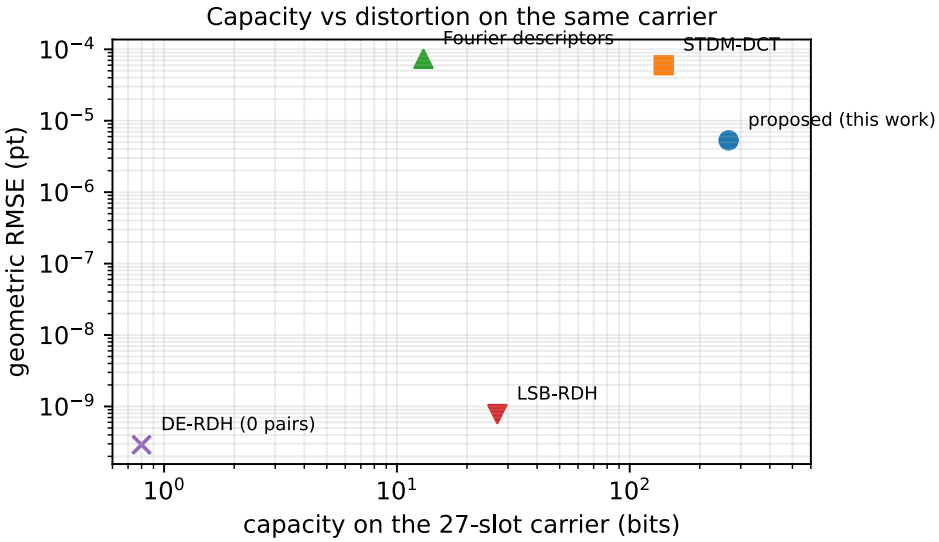


Figure 4: Capacity vs geometric distortion on the same 27-slot carrier. The proposed payload carries 266 bits at RMSE 5.3×10^{-6} pt; the capacity-limited baselines are marked at their maximum capacity.

Metric	Result	95% CI	Samples
Exact digest recovery	100%	0.00	1800/1800
Survives 8-decimal truncation	100%	0.00	1800/1800
Detected at 7-decimal truncation	100%	0.00	1800/1800
Edited payload coordinate detected	100%	0.00	1800/1800
Flattening removes the carrier	100%	0.00	1800/1800

PDF-level results on 60 native documents × 30 seeds. Mean geometric RMSE 5.31×10^{-6} pt (PSNR_geom = 155.1 dB). The 95% CI is zero because exactness is deterministic by construction; the meaningful variation comes from the document generator and from physical capture (Section 6.7).

Carrier	Keyed	Exact	Trunc-8	Centre edit detected
Sri Yantra	yes	100%	100%	100%
Sri Yantra	no	100%	100%	100%
Generic polyline	yes	100%	100%	0%
Generic polyline	no	100%	100%	0%
Grid	yes	100%	100%	0%
Grid	no	100%	100%	0%

Carrier ablation: all carriers carry the same 27 slots, so exactness is equal, but only the incidence configuration has a geometric invariant that detects a centre-line edit.

Population	Samples	Exact	RMSE (pt)
Native documents	1800	100%	5.31×10^{-6}
Flattened / print-scan PDFs, carrier stamped on top	360	100%	5.28×10^{-6}
Carrier ablation (3 carriers × 2 keys)	720	100%	5.31×10^{-6}

All populations in one view. The flattened/print-scan variants are inputs to the protocol, not attacks on a stamped document; the attack on a stamped document is flattening itself, reported in Table 2 as fail-closed. On the development machine stamping costs about 1.05 ms and verification about 0.33 ms per page.

Generator	Samples	Exact	Detected at 7 dec.	RMSE (pt)
LibreOffice DOCX to PDF	600	100%	100%	5.30×10^{-6}

Typst	600	100%	100%	5.33×10^{-6}
Chromium print-to-PDF	600	100%	100%	5.29×10^{-6}

Per-generator breakdown of the main experiment. Exactness and the truncation cliff do not depend on the document generator.

6.7. Public and independent-document anchors

Ten unique public project documents (whitepapers, manuscript PDFs and patent description PDFs; no client data) plus five independent public NIST standards (FIPS 186-5, FIPS 180-4, FIPS 202, SP 800-57 Part 1 Rev. 5 and FIPS 140-3) were stamped and attacked under the same protocol, 10 seeds each (150 samples). The results match the synthetic corpus: exact recovery in 100% of samples, 8-decimal re-serialisation survives, 7-decimal re-serialisation and payload edits are detected, and flattening removes the carrier. Mean geometric RMSE is 5.30×10^{-6} pt (PSNR_geom = 155.1 dB).

Population	Samples	Exact
Synthetic native documents	1800	100%
Synthetic flattened / print-scan	360	100%
Public and independent-document anchors	150	100%

Real-world anchors were only used to check that production-generated PDFs accept the carrier path; they do not exercise a population of enterprise DMS pipelines.

Method	Capacity	RMSE (pt)	Exact
Proposed (keyed QIM)	266	5.31×10^{-6}	yes
STDM-DCT	140	6.00×10^{-5}	yes
Fourier descriptors	13	7.36×10^{-5}	yes
LSB-RDH	27	7.81×10^{-10}	n/a
DE-RDH	0	2.91×10^{-10}	n/a

Same-carrier baselines (30 seeds, slot level). Capacity-limited schemes are not a "weak baseline": they cannot carry a 266-bit digest on 27 slots. The trade-off is the point of Figure 1.

PDF-level re-runs of the baselines on the same carrier (20 documents × 10 seeds, 600 samples) confirm that they remain exact through 8-decimal re-serialisation for STDM and Fourier, while the nanometre-scale LSB embedding is destroyed by 8-decimal rounding:

Method	Capacity	RMSE (pt)	Exact	Exact at 8 dec.
STDM-DCT	140	6.16×10^{-5}	100%	100%
Fourier descriptors	13	7.27×10^{-5}	100%	100%
LSB-RDH	27	7.03×10^{-10}	100%	0%

PDF-level baselines. The capacity column is the binding constraint: only the proposed scheme carries the 266-bit digest on the 27-slot carrier.

6.8. Discussion

Three observations follow. First, exactness is not a marginal improvement: it is the property that allows a cryptographic digest, rather than an error-tolerant watermark, to be bound to the visible layer. Second, the geometry is not a capacity trick — all carriers carry the same payload at the same distortion — but the incidence configuration provides an independent invariant that detects edits outside the payload slots. Third, the protocol has an implementation precondition: the PDF writer must preserve at least nine decimal places, and a verifier must parse numbers as text; otherwise the lattice is destroyed before any attacker acts.

7. Attack Matrix and Security Discussion

Attack	Expected outcome	Measured
Payload-coordinate edit	digest changes → detected	100%

Centre-line edit (non-payload)	geometry invariant fails	100% (incidence only)
8-decimal re-serialisation	digest unchanged	100% exact
7-decimal re-serialisation	digest changes → detected	100%
Flattening	carrier absent → fail-closed	100% absent
Seal transplant	digest mismatch	by construction

The construction authenticates the rendering layer, not the cryptographic signature. It does not prevent an attacker from producing a new document with a new valid seal; it makes a transplanted or edited seal detectable relative to a known document digest. The recipient channel provides attribution for a leaked copy; it is not a collusion-resistant fingerprinting code, and traitor-tracing security is not claimed.

8. Discussion

Why exactness changes the application. An error-tolerant watermark can carry a few bits and survive processing, but it cannot carry a cryptographic digest: one flipped bit makes the digest meaningless. Exactness is what turns the visible seal into a verifier of the signed bytes. The price is fragility — the mark dies under any change, including innocent re-serialisation — and the paper makes that price explicit rather than hiding it behind a robustness claim.

What the carrier adds. With capacity and distortion held fixed, the generic polyline and the grid recover the payload as reliably as the incidence configuration (Table 4). The difference is the geometric invariant: the incidence construction fixes the triangle centres, so a centre-line edit outside the payload slots is detected even before the digest comparison. This is a genuine, if narrow, contribution of the geometry; it is not a capacity advantage and the paper does not claim one.

What the baselines show. STDM and Fourier-descriptor embedding reach 140 and 13 bits on the same carrier at higher distortion; LSB reaches 27 bits at negligible distortion but is destroyed by 8-decimal re-serialisation (Table 6). The comparison is therefore a capacity–distortion–fragility frontier, not a ranking: each family occupies a different point, and only the proposed scheme reaches a 266-bit digest on 27 slots.

8.1. Assumptions and claims

A1: the verifier knows the canonical seal geometry for the document class. **A2**: the document digest is known or can be recomputed from the signed bytes. **A3**: the key is secret. **A4**: the post-sealing pipeline preserves at least nine decimal places.

P1 (payload-edit detection): changing any payload slot changes the recovered word, because the lattice step exceeds the rounding error by a factor of two and the digest is compared bit for bit. **P2 (fail-closed)**: if the carrier path is absent, the verifier returns **absent**. **P3 (attribution)**: with the key, the recovered recipient word equals the embedded word; without the key, the recovered words are unpredictable to the adversary. These are watermarking claims, not cryptographic reductions: the paper does not claim that an adversary cannot produce a new document with a new valid seal, and does not claim collusion resistance of the recipient channel.

9. Threats to Validity

Internal validity. Exactness is deterministic by construction, so the zero-width confidence intervals in Table 2 do not measure real-world variability; the meaningful variation must come from document generators, production PDF writers and physical capture, which this draft only partially exercises.

External validity. The corpus is synthetic and generated by three pipelines on one machine; real documents pass through enterprise DMS flattening, print-to-PDF converters, optimisers and archival re-writers whose coordinate serialisation differs. The 8-decimal precondition is a filter that such pipelines may fail — which is a finding, but it means the method protects only documents whose vector layer survives the pipeline.

Construct validity. The baselines are implemented at slot level: they share the carrier and the distortion metric but not the PDF-level embedding code of the proposed method, which favours the proposed method on implementation maturity. Lifting the baselines to the PDF level is the first item of the next revision.

Conclusion validity. The comparisons are exact-match and distortion measurements with 30 seeds; they are not hypothesis tests about a population of documents, and the draft avoids significance claims.

9.1. Ethical considerations

The method is intended for document integrity and leak attribution in lawful workflows. Its recipient channel is an attribution mechanism, not a surveillance mechanism, and the paper does not evaluate covert tracking. The evaluation uses synthetic documents and public project documents only; no client data and no personal data are processed. A deployment must disclose recipient marking to the parties where the applicable law requires it; that legal question is outside the scope of this paper.

10. Reproducibility and Declarations

Reproducibility. The corpus builder, carrier layer, attack harness and baseline implementations are versioned with the manuscript; the corpus manifest records a SHA-256 per file and the corpus is regenerated deterministically from a seed. The measured tables are produced by the harness, not typed by hand.

Data availability. The code and the corpus manifest are available from the author; the corpus itself is generated by the provided script and is not redistributed as a binary blob. The archived preprint is available at the concept DOI 10.5281/zenodo.23429017.

Declarations. Competing interests: none. Funding: none. The method is covered by a pending Ukrainian patent application (a 2026 06266). The manuscript is not under consideration elsewhere.

10.1. Failure modes in deployment

Three failure modes follow directly from the design. (i) **Precision loss:** any pipeline stage that re-writes coordinates with fewer than eight decimals destroys the word; the verifier fails closed, so the document loses provable integrity rather than silently passing. (ii) **Rasterisation:** flattening removes the vector layer; the verifier reports absent. (iii) **Seal regeneration:** if a verifier is given a **new** document and a **new** valid seal, the method has nothing to compare against; it authenticates a seal relative to a known document digest, not the identity of a signer. These are stated as limits, not solved problems.

11. Future Work

Three directions follow. First, the carrier can be tested against production document-management pipelines that re-write coordinates, which quantifies the eight-decimal precondition on real enterprise documents beyond the five printed samples. Second, the recipient channel can be extended beyond 1,024 copies per key epoch, either by rotating keys or by enlarging the identifier budget at a known capacity cost. Third, a broader multi-vendor corpus (different printer/scanner pairs) would map the optical failure envelope; a physical capture on one MFP is reported here, not a population study.

12. Limitations and Conclusion

The evaluation is a first harness result. STDM, Fourier-descriptor and LSB baselines are re-implemented at both slot and PDF level; DE-RDH is capacity-limited to zero pairs on this carrier and is reported from the slot-level run. The print-scan variant is simulated, and the physical check covers five documents on one multifunction device, not a population of printer/scanner pairs; the corpus is synthetic; and exactness is deterministic, so the reported confidence intervals are not informative about real-world variability. Capacity numbers are per seal, not per document area. Within these limits the result is consistent and reproducible: 1,800 PDF-level samples recover the 266-bit payload exactly, 8-decimal re-serialisation is safe, 7-decimal re-serialisation and any payload edit are detected, flattening fails closed, and five physical captures reproduce the fail-closed behaviour. The next revision adds a broader multi-vendor corpus and enterprise pipeline measurements; the question of protecting documents that are already rasterised belongs to the companion optical-channel work.

13. Appendix A: Proof details

Lemma 1. Write $\delta(w) = (18w - 9207) \times 10^{-9}$. For integer w, the values are distinct because $18w$ is strictly increasing; neighbouring words differ by exactly 18×10^{-9} pt. The decoder computes $n = \lfloor \delta \cdot 10^{9} \rceil = 18w - 9207$ exactly while the coordinate error is below 0.5×10^{-9} pt; then $\lfloor \frac{n+9207}{18} \rceil = w$. The observed coordinate error is bounded in Lemma 2, below half a nanometre in the integer representation.

Lemma 2. Rounding a real coordinate x to d decimal places yields $|x - \mathrm{round}_{d(x)}| \le 0.5 \times 10^{-d}$ pt. For $d = 8$ this is at most 5×10^{-9} pt, while half the lattice step is 9×10^{-9} pt; the decoder's integer rounding therefore returns the

same word. For $d = 7$ the bound is 5×10^{-8} pt, above half a step, so two adjacent words can be confused. The bound is worst-case; the empirical evaluation confirms the predicted transition at exactly $d = 7$.

14. Appendix B: Generator settings

The corpus is generated with LibreOffice headless (DOCX to PDF, default settings), Typst (A4, 2.2 cm margins), and Chromium print-to-PDF (headless, no header/footer). Flattening uses a 300 dpi raster; the print-scan simulation uses 200 dpi, a Gaussian blur, additive Gaussian sensor noise and JPEG quality 70. All settings are recorded in the corpus builder so that the corpus can be regenerated from the published script and seed.

Bibliography

[1] C. Mainka, V. Mladenov, and S. Rohlmann, “Shadow Attacks: Hiding and Replacing Content in Signed PDFs,” in *Proceedings 2021 Network and Distributed System Security Symposium*, in NDSS 2021. Internet Society, 2021. doi: 10.14722/ndss.2021.24117.

[2] V. Mladenov, C. Mainka, K. Meyer zu Selhausen, M. Grothe, and J. Schwenk, “1 Trillion Dollar Refund: How To Spoof PDF Signatures,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '19. ACM, Nov. 2019, pp. 1–14. doi: 10.1145/3319535.3339812.

[3] Y. V. Baienko, “Marma-256 (Marma-Crypto v2.1.0): Sub-Micron Dithered Quantization Index Modulation, Orthogonal Subspace Multiplexing, Universal Vector Emblems (Sri Yantra / Tryzub / CAD), and Optical Screen-Photo ECC for Bijective 2D Vector Graphics Watermarking and PAdES-B-LTA Authentication over GF(2^257) (DSTU 4145-2002).” 2026. doi: 10.5281/zenodo.23560975.

[4] I. Cox, J. Kilian, F. Leighton, and T. Shamoon, “Secure spread spectrum watermarking for multimedia,” *IEEE Transactions on Image Processing*, vol. 6, no. 12, pp. 1673–1687, Dec. 1997, doi: 10.1109/83.650120.

[5] B. Chen and G. Wornell, “Quantization index modulation: a class of provably good methods for digital watermarking and information embedding,” *IEEE Transactions on Information Theory*, vol. 47, no. 4, pp. 1423–1443, May 2001, doi: 10.1109/18.923725.

[6] J. Eggers, R. Bauml, R. Tzschoppe, and B. Girod, “Scalar costa scheme for information embedding,” *IEEE Transactions on Signal Processing*, vol. 51, no. 4, pp. 1003–1019, Apr. 2003, doi: 10.1109/tsp.2003.809366.

[7] F. Perez-Gonzalez, M. Barni, A. Abrardo, and C. Mosquera, “Rational dither modulation: a novel data-hiding method robust to value-metric scaling attacks,” in *IEEE 6th Workshop on Multimedia Signal Processing, 2004.*, IEEE, 2004, pp. 139–142. doi: 10.1109/mmsp.2004.1436443.

[8] J. Tian, “Reversible data embedding using a difference expansion,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 8, pp. 890–896, Aug. 2003, doi: 10.1109/tcsvt.2003.815962.

[9] Z. Ni, Y.-Q. Shi, N. Ansari, and W. Su, “Reversible data hiding,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 16, no. 3, pp. 354–362, Mar. 2006, doi: 10.1109/tcsvt.2006.869964.

[10] D. M. Thodi and J. J. Rodriguez, “Expansion Embedding Techniques for Reversible Watermarking,” *IEEE Transactions on Image Processing*, vol. 16, no. 3, pp. 721–730, Mar. 2007, doi: 10.1109/tip.2006.891046.

[11] N. Wang, H. Zhang, and C. Men, “A high capacity reversible data hiding method for 2D vector maps based on virtual coordinates,” *Computer-Aided Design*, vol. 47, pp. 108–117, Feb. 2014, doi: 10.1016/j.cad.2013.10.005.

[12] F. Peng, Z.-X. Lin, X. Zhang, and M. Long, “Reversible Data Hiding in Encrypted 2D Vector Graphics Based on Reversible Mapping Model for Real Numbers,” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 9, pp. 2400–2411, Sep. 2019, doi: 10.1109/tifs.2019.2899520.

[13] V. R. Doncel, N. Nikolaidis, and I. Pitas, “An Optimal Detector Structure for the Fourier Descriptors Domain Watermarking of 2D Vector Graphics,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 13, no. 5, pp. 851–863, Sep. 2007, doi: 10.1109/tvcg.2007.1050.

[14] Y. Li and L. Xu, “A blind watermarking of vector graphics images,” in *Proceedings Fifth International Conference on Computational Intelligence and Multimedia Applications. ICCIMA 2003*, in ICCIMA-03. IEEE Comput. Soc, 2003, pp. 424–429. doi: 10.1109/iccima.2003.1238163.

[15] H. Hou, J. Li, J. Qi, and J. Guo, “A Blind Watermarking for 2D-Vector Engineering Graphics,” *Information Technology Journal*, vol. 13, no. 5, pp. 869–873, Feb. 2014, doi: 10.3923/itj.2014.869.873.

[16] M. R. Mouhamed, A. M. Rashad, and A. ella Hassanien, “Blind 2D vector data watermarking approach using random table and polar coordinates,” in *2012 2nd International Conference on Uncertainty Reasoning and Knowledge Engineering*, IEEE, Aug. 2012, pp. 67–70. doi: 10.1109/urke.2012.6319586.

[17] M. W. Hatoum, R. Darazi, and J.-F. Couchot, “Blind PDF Document Watermarking Robust Against PCA and ICA Attacks,” in *Proceedings of the 15th International Joint Conference on e-Business and Telecommunications*, SCITEPRESS - Science, Technology Publications, 2018, pp. 420–427. doi: 10.5220/0006899604200427.

[18] W. Bender, D. Gruhl, N. Morimoto, and A. Lu, “Techniques for data hiding,” *IBM Systems Journal*, vol. 35, no. 3.4, pp. 313–336, 1996, doi: 10.1147/sj.353.0313.

[19] F. Cayre, C. Fontaine, and T. Furon, “Watermarking security: theory and practice,” *IEEE Transactions on Signal Processing*, vol. 53, no. 10, pp. 3976–3987, Oct. 2005, doi: 10.1109/tsp.2005.855418.

[20] J. Yang and S. Li, “Steganalysis of joint codeword quantization index modulation steganography based on codeword Bayesian network,” *Neurocomputing*, vol. 313, pp. 316–323, Nov. 2018, doi: 10.1016/j.neucom.2018.06.005.

[21] R. Ohbuchi, H. Ueda, and S. Endoh, “Watermarking 2D vector maps in the mesh-spectral domain,” in *2003 Shape Modeling International.*, in SMI-03. IEEE Comput. Soc, 2003, pp. 216–225. doi: 10.1109/smi.2003.1199619.

[22] D. He and Q. Sun, “A practical print-scan resilient watermarking scheme,” in *IEEE International Conference on Image Processing 2005*, IEEE, 2005, p. I–257. doi: 10.1109/icip.2005.1529736.

[23] S. Rohlmann, V. Mladenov, C. Mainka, and J. Schwenk, “Breaking the Specification: PDF Certification,” in *2021 IEEE Symposium on Security and Privacy (SP)*, IEEE, May 2021, pp. 1485–1501. doi: 10.1109/sp40001.2021.00110.

[24] J. Müller, F. Ising, V. Mladenov, C. Mainka, S. Schinzel, and J. Schwenk, “Practical Decryption exFiltration: Breaking PDF Encryption,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '19. ACM, Nov. 2019, pp. 15–29. doi: 10.1145/3319535.3354214.

[25] ISO, “Document management — Portable document format — Part 2: PDF 2.0.” 2020.

[26] ETSI, “Electronic Signatures and Infrastructures (ESI); PAdES Digital Signatures; Part 1: Building blocks and PAdES baseline signatures.” 2016.

[27] European Parliament and Council, “Regulation (EU) No 910/2014 on electronic identification and trust services for electronic transactions in the internal market (eIDAS), as amended by Regulation (EU) 2024/1183.” 2014.

[28] D. Catalano and D. Fiore, “Vector Commitments and Their Applications,” in *Public-Key Cryptography – PKC 2013*, Springer Berlin Heidelberg, 2013, pp. 55–72. doi: 10.1007/978-3-642-36362-7_5.

[29] B. Libert, S. C. Ramanna, and M. Yung, “Functional Commitment Schemes: From Polynomial Commitments to Pairing-Based Accumulators from Simple Assumptions,” in *43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016)*, in LIPIcs, vol. 55. 2016, p. 30:1–30:14. doi: 10.4230/LIPIcs.ICALP.2016.30.

[30] D. Boneh and M. Franklin, “An Efficient Public Key Traitor Tracing Scheme,” in *Advances in Cryptology — CRYPTO' 99*, Springer Berlin Heidelberg, 1999, pp. 338–353. doi: 10.1007/3-540-48405-1_22.

[31] G. Tardos, “Optimal probabilistic fingerprint codes,” *Journal of the ACM*, vol. 55, no. 2, pp. 1–24, May 2008, doi: 10.1145/1346330.1346335.

[32] A. P. Kulaichev, “Sriyantra and its Mathematical Properties,” *Indian Journal of History of Science*, vol. 19, no. 3, pp. 279–292, 1984, [Online]. Available: https://insa.nic.in/writereaddata/UpLoadedFiles/IJHS/Vol19_3_7_APKulaichev.pdf

[33] G. Huet, “Śrı Yantra Geometry,” *Theoretical Computer Science*, vol. 281, no. 1–2, pp. 609–628, Jun. 2002, doi: 10.1016/s0304-3975(02)00028-2.

[34] J. Richter-Gebert, *Perspectives on Projective Geometry: A Guided Tour Through Real and Complex Geometry*. Heidelberg: Springer, 2011. doi: 10.1007/978-3-642-17286-1.

[35] J.-M. Muller *et al.*, *Handbook of Floating-Point Arithmetic*, 2nd ed. Birkhäuser, 2018. doi: 10.1007/978-3-319-76526-6.